Light

Algorithms — Trace Tables & Dry Runs

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Understanding algorithms
2. Trace tables
3. Standard algorithms
Worked examples
Quick reference
Typical pitfalls

Edexcel GCSE Computer Science — **Algorithms and Trace Tables (1CP2/01 Topic 1: Computational Thinking)**. This topic covers understanding what algorithms are, following and tracing algorithms using trace tables, and the four standard algorithms: linear search, binary search, bubble sort, and merge sort.

Exam questions typically ask you to **follow** algorithms and complete trace tables, **state** the output of algorithms, **describe** what algorithms do, and **compare** the efficiency of different algorithms. Paper 1 questions typically range from 2—6 marks. Paper 2 requires you to write and trace Python code in practical programming tasks.

Why algorithms and trace tables matter

An algorithm is a step-by-step set of instructions to solve a problem or complete a task. Understanding algorithms is fundamental to computer science:

- **Problem solving:** Algorithms provide systematic approaches to solving problems
- **Programming foundation:** All programs are implementations of algorithms
- **Efficiency:** Understanding different algorithms helps you choose the most efficient solution
- **Debugging:** Trace tables help you follow code execution and find errors

- **Exam relevance:** Topic 1 typically carries 15—20 marks across Paper 1, with trace table and algorithm questions appearing in most exams

What you must know

This checklist covers the key exam-visible concepts from Edexcel 1CP2 Topic 1 relating to algorithms and trace tables.

- **Define** what an algorithm is and explain what algorithms are used for
- **Describe** what a given algorithm does when presented with flowcharts, pseudocode, or program code
- **Follow** algorithms and trace the values of variables at each step using a trace table
- **State** the output of an algorithm at a specific point or after completion
- **Explain** the purpose and operation of the four standard algorithms: linear search, binary search, bubble sort, and merge sort
- **Compare** the efficiency of searching and sorting algorithms in terms of comparisons and passes
- **Amend** algorithms to change their functionality or fix errors

> **Specification link:** This topic covers Edexcel 1CP2 Paper 1 Topic 1.2 (Algorithmic thinking) — understanding what algorithms are, following and tracing algorithms, and the four standard algorithms (linear search, binary search, bubble sort, merge sort).

Toolkit

These are the key patterns, structures, and templates you need for exam questions about algorithms and trace tables.

Algorithm representations

Algorithms can be expressed in four ways. You must be able to follow all of them:

1. Written description

Plain English steps describing what to do:

1. Set total to 0
2. For each number in the list
3. Add the number to total
4. Output total

2. Pseudocode

Structured English that looks like code:

```
total ← 0
FOR each number IN list
    total ← total + number
ENDFOR
OUTPUT total
```

3. Flowchart

Visual diagram using standard symbols:

- **Oval:** Start / Stop
- **Rectangle:** Process (action)
- **Parallelogram:** Input / Output
- **Diamond:** Decision (Yes/No)
- **Arrow:** Flow direction
- **Rectangle with lines:** Subprogram

4. Program code (Python)

Actual code that can be executed:

```
total = 0
for number in list:
    total = total + number
print(total)
```

Trace table structure

A trace table records the values of variables as an algorithm executes. Each row represents one step or iteration:

Step	Variable 1	Variable 2	Condition	Output
1	initial value	initial value	—	—
2	new value	unchanged	True/False	—
...	...	...	...	value

Exam tip: When completing a trace table, work through the algorithm line by line. Only update values when they change. Use "—" or leave blank for values that don't change in a step.

Standard algorithm patterns

Algorithm	Purpose	Requires sorted data?	Key characteristic
Linear search	Find an item in a list	No	Checks each item in order
Binary search	Find an item in a list	Yes	Halves the search space each time

Algorithm	Purpose	Requires sorted data?	Key characteristic
Bubble sort	Sort a list into order	N/A (sorts data)	Compares adjacent pairs, swaps if needed
Merge sort	Sort a list into order	N/A (sorts data)	Divides list, sorts halves, merges

Key Python constructs for tracing

Count-controlled loop

```
for i in range(5):
    # i takes values 0, 1, 2, 3, 4
    print(i)

for i in range(2, 7):
    # i takes values 2, 3, 4, 5, 6
    print(i)
```

Condition-controlled loop

```
count = 0
while count < 5:
    print(count)
    count = count + 1
# Loops while condition is True
```

Selection (if-elif-else)

```
if score >= 50:
    print("Pass")
elif score >= 40:
    print("Near miss")
else:
    print("Fail")
```

List indexing

```
data = [10, 20, 30, 40, 50]
# Index:  0   1   2   3   4

print(data[0])  # Output: 10
print(data[2])  # Output: 30
print(len(data))  # Output: 5
```

1. Understanding algorithms

An **algorithm** is a precise sequence of instructions that describes how to solve a problem or complete a task. Every computer program is an implementation of one or more algorithms.

1.1 What is an algorithm?

Algorithms have several key characteristics:

- **Finite:** Must have a defined end point — it cannot run forever
- **Precise:** Each step must be unambiguous and clearly defined
- **Effective:** Each step must be achievable
- **Input:** Takes zero or more inputs
- **Output:** Produces at least one output

Real-world examples of algorithms

- A recipe for baking a cake
- Directions to get from A to B
- Instructions for assembling furniture
- The steps to withdraw money from an ATM

> **Exam tip:** When asked to "describe what an algorithm does", explain the overall purpose and then outline the main steps it follows to achieve that purpose.

1.2 Following algorithms

To follow an algorithm means to work through it step by step, tracking what happens at each stage. This is essential for:

- Understanding what the algorithm does
- Predicting the output for given inputs
- Finding errors in the algorithm
- Verifying the algorithm works correctly

Steps to follow an algorithm

1. Identify all variables and their initial values
2. Work through each instruction in order
3. Update variable values as they change
4. Follow the correct path at decision points
5. Record any outputs produced

Key skill: Being able to follow an algorithm is tested in both Paper 1 (following flowcharts and pseudocode) and Paper 2 (tracing Python code and predicting outputs).

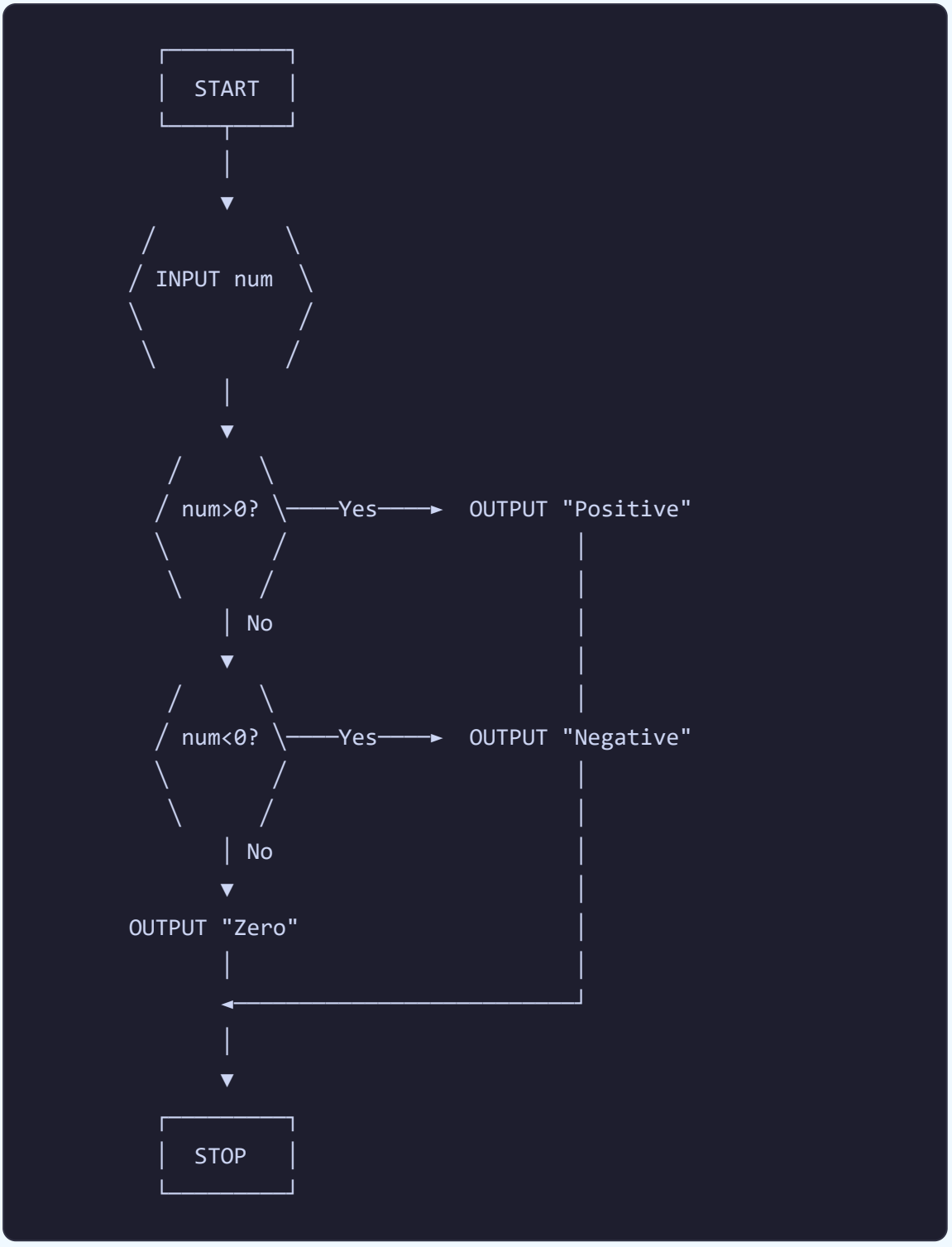
1.3 Flowchart symbols

Edexcel provides standard flowchart symbols in Appendix 2 of the specification. You must recognise and use these symbols:

```
FLOWCHART SYMBOLS (Edexcel Appendix 2)

  ┌──────────────┐
  │  START / STOP │    Oval — start and end of algorithm
  └──────────────┘

  ┌──────────────┐
  │    PROCESS   │    Rectangle — action or calculation
  └──────────────┘    e.g. total = total + 1

   /            \
  / INPUT/OUTPUT  \   Parallelogram — input or output
  \               /   e.g. INPUT name, OUTPUT result
   \            /

      /\
     /  \
    / ?? \            Diamond — decision (Yes/No)
    \    /            e.g. Is x > 10?
     \  /
      \/

  ┌─┬──────────┬─┐
  │ │ SUBPROG  │ │    Rectangle with lines — subprogram
  └─┴──────────┴─┘

       |
       ▼            Arrow — flow direction
       |
```

Example flowchart

This flowchart shows an algorithm that outputs whether a number is positive, negative, or zero:



Worked Example 1 — Describing an algorithm's purpose

Question: The following pseudocode shows an algorithm. Describe what this algorithm does. [2 marks]

```
total ← 0
count ← 0
```

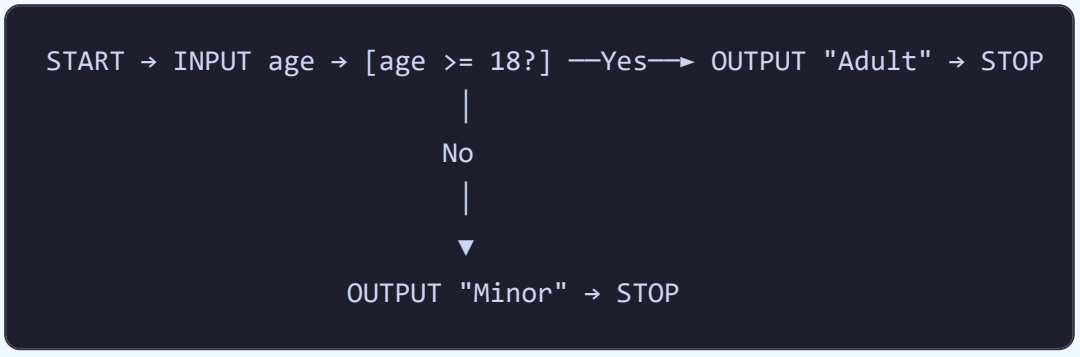
```
FOR i ← 1 TO 5
    INPUT number
    total ← total + number
    count ← count + 1
ENDFOR
average ← total / count
OUTPUT average
```

Model answer:
The algorithm calculates the average of 5 numbers **(1)**.
It inputs 5 numbers, adds them together to get a total, then divides the total by the count (5) to calculate and output the average **(1)**.

Exam tip: Start with the overall purpose, then briefly explain how it achieves this. Don't describe every line — focus on what the algorithm accomplishes.

Worked Example 2 – Following a flowchart

Question: The flowchart below checks if a user is old enough. If the input is age = 16, state the output. [1 mark]



Model answer:
Output: **"Minor" (1)**
Working: age = 16, so age >= 18 is False (16 is not greater than or equal to 18), so the "No" path is followed.

Exam tip: Always substitute the actual values into conditions. 16 >= 18 is False, so follow the "No" branch.

Worked Example 3 — Determining variable values

Question: What is the value of `x` after this code executes? [1 mark]

```
x = 5
y = 3
x = x + y
y = x - y
x = x - y
```

Model answer:
x = **3 (1)**
Working through step by step:

- Line 1: x = 5
- Line 2: y = 3
- Line 3: x = 5 + 3 = 8
- Line 4: y = 8 - 3 = 5
- Line 5: x = 8 - 5 = 3

Exam tip: This is a classic swap algorithm pattern. Always use the current values of variables, not the original values.

Worked Example 4 — Counting loop iterations

Question: How many times does this loop execute? What is the final value of `total`? [2 marks]

```
total = 0
for i in range(1, 6):
    total = total + i
```

Model answer:
The loop executes **5 times (1)** (i takes values 1, 2, 3, 4, 5).
Final value of total = **15 (1)** (0 + 1 + 2 + 3 + 4 + 5 = 15).

Exam tip: `range(1, 6)` includes 1 but excludes 6, so it generates 1, 2, 3, 4, 5. The loop runs 5 times, not 6.

2. Trace tables

A **trace table** is a technique used to track the values of variables as an algorithm executes. Each row represents a step or iteration, showing how values change over time.

Trace tables are essential for understanding algorithm behaviour and finding errors.

2.1 Creating trace tables

To create a trace table:

1. **Identify all variables** — create a column for each variable
2. **Add condition/output columns** — if the algorithm has decisions or outputs
3. **Work through line by line** — update only the values that change
4. **Add a new row** — each time you complete a step or iteration
5. **Record the final state** — when the algorithm terminates

Common column headings

- Variable names (x, total, count, i, etc.)
- Array/list values with indices
- Condition results (True/False)
- Output values
- Line number (optional)

2.2 When to add rows

Different conventions exist, but the most common approach is:

- **One row per loop iteration** — showing the state at the end of each pass
- **One row per significant change** — when any variable is modified
- **One row per line executed** — for detailed tracing

> **Exam guidance:** The exam may specify "complete the trace table for each pass through the loop" or "show the value after each iteration". Follow the instructions given. If no guidance, one row per iteration is usually expected.

Notation in trace tables

- Use actual values (5, "Hello", True)
- Use "—" or leave blank for unchanged values
- Show array contents in square brackets: [3, 7, 2]
- For conditions, write True/False or Yes/No

2.3 Example: Tracing a simple loop

Consider this algorithm that counts down from 5 to 1:

```
count = 5
while count > 0:
    print(count)
    count = count - 1
```

The trace table for this algorithm:

Step	count	count > 0	Output
1	5	True	5
2	4	True	4
3	3	True	3
4	2	True	2

Step	count	count > 0	Output
5	1	True	1
6	0	False	–

The loop terminates when count becomes 0 because the condition `count > 0` is False.

2.4 Tracing with arrays/lists

When tracing algorithms that modify lists, show the complete list state after each change:

```
data = [10, 20, 30]
for i in range(len(data)):
    data[i] = data[i] * 2
```

Iteration	i	data[i] before	data[i] after	data (full list)
1	0	10	20	[20, 20, 30]
2	1	20	40	[20, 40, 30]
3	2	30	60	[20, 40, 60]

Worked Example 5 — Completing a trace table

Question: Complete the trace table for the following algorithm when the input is 4. [3 marks]

```
INPUT n
result = 1
FOR i = 1 TO n
    result = result * i
ENDFOR
OUTPUT result
```

Model answer:

i	result
–	1
1	1
2	2
3	6
4	24

Output: **24**

This algorithm calculates n factorial (n!). 4! = 1 × 1 × 2 × 3 × 4 = 24

Exam tip: The loop runs from i = 1 TO 4, so 4 iterations. result starts at 1, then becomes 1×1=1, 1×2=2, 2×3=6, 6×4=24.

Worked Example 6 — Tracing a while loop

Question: Complete the trace table for this algorithm. State the final output. [4 marks]

```
x = 100
count = 0
while x > 1:
    x = x // 2
    count = count + 1
print(count)
```

Model answer:

Iteration	x	count	x > 1
Start	100	0	True
1	50	1	True
2	25	2	True
3	12	3	True
4	6	4	True
5	3	5	True
6	1	6	False

Output: **6**

Exam tip: The // operator is integer division (divides and rounds down). 25 // 2 = 12, not 12.5. This algorithm counts how many times you can halve a number before reaching 1.

Worked Example 7 — Tracing with list modification

Question: Trace this algorithm with input data = [5, 2, 8, 1]. Show the list after each iteration of the outer loop. [3 marks]

```
data = [5, 2, 8, 1]
for i in range(len(data)):
    if data[i] > 5:
        data[i] = 5
```

Model answer:

i	data[i]	data[i] > 5	data after
0	5	False	[5, 2, 8, 1]
1	2	False	[5, 2, 8, 1]
2	8	True	[5, 2, 5, 1]
3	1	False	[5, 2, 5, 1]

The algorithm caps any value greater than 5 to 5. Only data[2] (value 8) is changed.

Exam tip: When the condition is False, no change occurs. Show the list state anyway to demonstrate you checked the condition.

Worked Example 8 — Finding the output from a trace

Question: What is output by this algorithm? [2 marks]

```
word = "COMPUTING"
result = ""
for i in range(0, len(word), 2):
    result = result + word[i]
print(result)
```

Model answer:

Output: **CMUIG (2)**

Working — write out the string with indices:

```
Index:   0   1   2   3   4   5   6   7   8
Char:    C   O   M   P   U   T   I   N   G
```

- range(0, 9, 2) generates indices: 0, 2, 4, 6, 8
- word[0]='C', word[2]='M', word[4]='U', word[6]='I', word[8]='G'
- result builds: "C" → "CM" → "CMU" → "CMUI" → "CMUIG"

> **Exam tip:** range(start, stop, step) with step=2 takes every second index. Always write out the string with indices underneath to avoid errors.

3. Standard algorithms

Edexcel specifies four standard algorithms you must understand: two searching algorithms (linear search, binary search) and two sorting algorithms (bubble sort, merge sort). You need to understand how they work, trace their execution, and compare their efficiency.

3.1 Linear search

Linear search finds an item by checking each element in order, from the first to the last, until the item is found or the end is reached.

How it works

1. Start at the first element
2. Compare the current element to the target
3. If they match, return the position
4. If not, move to the next element
5. Repeat until found or end of list reached

Python implementation

```python
def linear_search(data, target):
    for i in range(len(data)):
        if data[i] == target:
            return i
    return -1  # Not found
```

Key characteristics

- Works on unsorted and sorted data
- Simple to implement
- Best case: 1 comparison (item is first)
- Worst case: n comparisons (item is last or not present)
- Average case: n/2 comparisons

3.2 Binary search

Binary search finds an item by repeatedly dividing the search space in half. It requires the data to be sorted.

How it works

1. Find the middle element
2. If middle equals target, return position
3. If target is less than middle, search left half
4. If target is greater than middle, search right half
5. Repeat until found or search space is empty

Python implementation

```python
def binary_search(data, target):
    low = 0
```

```
    high = len(data) - 1
    while low <= high:
        mid = (low + high) // 2
        if data[mid] == target:
            return mid
        elif data[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1  # Not found
```

Key characteristics

- **Requires sorted data**
- More complex to implement
- Much faster for large datasets
- Worst case: $\log_2(n)$ comparisons
- For 1000 items: max 10 comparisons

Comparing search algorithms

Feature	Linear search	Binary search
Requires sorted data?	No	Yes
Best case comparisons	1	1
Worst case comparisons	n	$\log_2(n)$
For 1000 items (worst)	1000	10
For 1,000,000 items (worst)	1,000,000	20
Complexity	Simple	More complex

3.3 Bubble sort

Bubble sort repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. Larger values "bubble" to the end.

How it works

1. Compare adjacent pairs from the start
2. Swap if left element is greater than right
3. Continue to end of list (one pass)
4. Largest element is now at the end
5. Repeat, excluding the last sorted element
6. Stop when no swaps occur in a pass

Python implementation

```
def bubble_sort(data):
    n = len(data)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if data[j] > data[j + 1]:
```

```
                # Swap
                temp = data[j]
                data[j] = data[j + 1]
                data[j + 1] = temp
    return data
```

Key characteristics

- Simple to understand and implement
- Works in place (no extra memory)
- Inefficient for large datasets
- Maximum n-1 passes needed
- Worst case: n(n-1)/2 comparisons

3.4 Merge sort

Merge sort uses a divide-and-conquer approach: it splits the list into halves, sorts each half, then merges the sorted halves back together.

> **GCSE note:** At GCSE, you need to understand how merge sort works conceptually and be able to trace through examples. You do not need to write the full code implementation.

How it works

1. If the list has 0 or 1 elements, it's already sorted
2. Split the list into two halves
3. Sort each half (repeat steps 1—2 on each half)
4. Merge the two sorted halves:
   - Compare first elements of each half
   - Take the smaller one
   - Repeat until both halves are empty

Visual example

```
Original:   [38, 27, 43, 3, 9, 82, 10]
                      |
          ┌───────────┴───────────┐
          ▼                       ▼
  [38, 27, 43, 3]         [9, 82, 10]
        |                       |
    ┌───┴───┐               ┌───┴───┐
    ▼       ▼               ▼       ▼
[38,27]   [43,3]          [9,82]    [10]
   |        |               |        |
  ┌┴┐      ┌┴┐             ┌┴┐       |
  ▼ ▼      ▼ ▼             ▼ ▼       ▼
[38][27] [43] [3]         [9]  [82]   [10]

Now merge back up:
```

```
[27,38]   [3,43]          [9,82]     [10]
   |         |              |         |
   └─────┬───┘              └────┬────┘
         ▼                       ▼
  [3, 27, 38, 43]          [9, 10, 82]
         |                        |
         └────────────┬───────────┘
                      ▼
          [3, 9, 10, 27, 38, 43, 82]
```

Key characteristics

- More complex to understand than bubble sort
- Requires additional memory for the sub-lists
- Much faster for large datasets
- Consistent performance regardless of initial data order
- Always approximately n × $\log_2(n)$ comparisons

Comparing sorting algorithms

Feature	Bubble sort	Merge sort
Method	Compare adjacent pairs, swap	Divide, sort halves, merge
Best case	n comparisons (already sorted)	n × $\log_2(n)$ comparisons
Worst case	n^2 comparisons	n × $\log_2(n)$ comparisons
Extra memory	None (in place)	Requires extra space
For 1000 items (worst)	~500,000 comparisons	~10,000 comparisons
Best for	Small or nearly-sorted lists	Large datasets

Worked Example 9 — Tracing linear search

Question: Trace the linear search algorithm to find the value 7 in the list [3, 9, 7, 2, 5]. State the number of comparisons made. [3 marks]

Model answer:

Comparison	Index (i)	data[i]	data[i] == 7?
1	0	3	False
2	1	9	False
3	2	7	True — Found!

Number of comparisons: **3**
The algorithm returns index **2** (the position where 7 is found).

Exam tip: Linear search stops as soon as the item is found. It doesn't continue checking the rest of the list.

Worked Example 10 — Tracing binary search

Question: Trace the binary search algorithm to find the value 35 in the sorted list [5, 11, 17, 23, 29, 35, 41]. Show the values of low, high, and mid at each step. [4 marks]

Model answer:

Step	low	high	mid	data[mid]	Comparison	Action
1	0	6	3	23	35 > 23	Search right: low = 4
2	4	6	5	35	35 == 35	Found at index 5

Number of comparisons: **2**
The algorithm returns index **5**.
Working: mid = (0+6)//2 = 3. data[3]=23. Since 35 > 23, search right half. New mid = (4+6)//2 = 5. data[5]=35. Found!

Exam tip: mid = (low + high) // 2. After each comparison, either low or high is updated to narrow the search space by half. Always show your working for the mid calculation.

Worked Example 11 — Tracing bubble sort (one pass)

Question: Show the state of the list [6, 2, 9, 3, 5] after one complete pass of bubble sort. How many swaps occurred? [3 marks]

Model answer:

Compare	Pair	Swap?	List after
1	6 and 2	Yes (6 > 2)	[2, 6, 9, 3, 5]
2	6 and 9	No (6 < 9)	[2, 6, 9, 3, 5]
3	9 and 3	Yes (9 > 3)	[2, 6, 3, 9, 5]
4	9 and 5	Yes (9 > 5)	[2, 6, 3, 5, 9]

List after first pass: **[2, 6, 3, 5, 9]**
Number of swaps: **3**
Notice the largest value (9) has "bubbled" to the end.

Exam tip: After each pass of bubble sort, the largest unsorted element is guaranteed to be in its correct position. After pass 1, the last element is sorted; after pass 2, the last two are sorted, etc.

Worked Example 12 – Comparing algorithm efficiency

Question: A list contains 1000 items. Compare the maximum number of comparisons needed to find an item using linear search versus binary search. Which is more efficient and why? [4 marks]

Model answer:
Linear search: Maximum 1000 comparisons **(1)** — in the worst case, the item is at the end or not present, so every element must be checked.
Binary search: Maximum 10 comparisons **(1)** — each comparison halves the search space. $2^{10} = 1024$, so 10 comparisons can search up to 1024 items.
Binary search is more efficient (1) because it eliminates half of the remaining items with each comparison, rather than just one item at a time **(1)**.
However, binary search requires the data to be sorted first.

Exam tip: For "compare" questions, state specific numbers where possible. $\log_2(1000) \approx 10$, so binary search needs at most 10 comparisons for 1000 items.

Worked examples summary

The table below summarises the 12 worked examples covered in this topic, organised by the type of algorithm skill they assess.

Example	Skill assessed	Key technique
1	Describing algorithm purpose	State overall purpose, then main steps
2	Following a flowchart	Substitute values, follow correct branch
3	Determining variable values	Work through line by line with current values
4	Counting loop iterations	Understand range() parameters
5	Completing trace tables	One row per iteration, update values

Example	Skill assessed	Key technique
6	Tracing while loops	Continue until condition is False
7	Tracing with list modification	Show full list state after each change
8	String processing output	Write indices under string characters
9	Tracing linear search	Check each element in order until found
10	Tracing binary search	Calculate mid, update low/high each step
11	Tracing bubble sort	Compare adjacent pairs, swap if needed
12	Comparing algorithm efficiency	Use specific numbers and explain why

Quick reference

Key definitions

Term	Definition
Algorithm	A precise sequence of instructions to solve a problem or complete a task
Trace table	A table that records the values of variables as an algorithm executes
Linear search	A search algorithm that checks each item in order until the target is found
Binary search	A search algorithm that repeatedly halves the search space by comparing to the middle element (requires sorted data)
Bubble sort	A sorting algorithm that repeatedly compares adjacent pairs and swaps them if in the wrong order
Merge sort	A sorting algorithm that divides the list into halves, sorts each half, then merges them back together
Pass	One complete iteration through a list in a sorting algorithm
Comparison	A single check to see if one value is equal to, greater than, or less than another

Flowchart symbols

Symbol	Shape	Purpose
Terminator	Oval / Rounded rectangle	Start and end of algorithm

Symbol	Shape	Purpose
Process	Rectangle	An action or calculation
Input/Output	Parallelogram	Data input or output
Decision	Diamond	Yes/No question (branching)
Flow line	Arrow	Direction of flow
Subprogram	Rectangle with vertical lines	Call to a pre-defined procedure

Algorithm efficiency summary

Algorithm	Best case	Worst case	Sorted data needed?
Linear search	1	n	No
Binary search	1	$\log_2(n)$	Yes
Bubble sort	n	n^2	N/A
Merge sort	$n \log_2(n)$	$n \log_2(n)$	N/A

Powers of 2 (for binary search)

Power	Value	Binary search: max items with this many comparisons
2^1	2	2
2^2	4	4
2^3	8	8
2^4	16	16
2^5	32	32
2^{10}	1,024	~1,000
2^{20}	1,048,576	~1,000,000

Typical pitfalls

These are common mistakes students make in algorithm and trace table questions. Avoiding these errors will help you gain marks in the exam.

Pitfall 1 — Off-by-one errors with range().
Wrong: Thinking range(5) gives 1, 2, 3, 4, 5.

Right: range(5) gives **0, 1, 2, 3, 4**. It starts at 0 and excludes the end value. range(1, 6) gives 1, 2, 3, 4, 5.

Pitfall 2 — Using original values instead of current values.
Wrong: After x = x + 5, using the original x value in the next line.
Right: Always use the **current** value of a variable, which may have changed in a previous line.

Pitfall 3 — Forgetting binary search needs sorted data.
Wrong: "Binary search is always better than linear search."
Right: Binary search only works on **sorted data**. If data is unsorted, you must use linear search or sort it first.

Pitfall 4 — Confusing // and / in Python.
Wrong: Thinking 7 // 2 = 3.5.
Right: // is **integer division** (rounds down). 7 // 2 = **3**. Regular / gives 3.5.

Pitfall 5 — List index errors.
Wrong: A list of 5 items has indices 1 to 5.
Right: Python lists are **zero-indexed**. A list of 5 items has indices **0 to 4**.

Pitfall 6 — Not showing the termination condition.
Wrong: Stopping a trace table when the answer is found.
Right: For while loops, show the final row where the **condition becomes False** and the loop exits.

Pitfall 7 — Bubble sort pass count.
Wrong: "Bubble sort needs n passes to sort n items."
Right: Bubble sort needs at most **n - 1** passes. After n - 1 passes, all elements are guaranteed to be sorted.

Pitfall 8 — Binary search mid calculation.
Wrong: Calculating mid as (high - low) / 2.
Right: mid = **(low + high) // 2**. This gives the correct middle index between low and high.

Pitfall 9 — Forgetting to update loop variables.
Wrong: Infinite loop because the variable controlling the while condition is never changed.
Right: Ensure the loop variable is **updated inside the loop** so the condition eventually becomes False.

Pitfall 10 — Vague efficiency comparisons.
Wrong: "Binary search is faster."
Right: Give **specific numbers**: "Binary search needs at most 10 comparisons for 1000 items, while linear search needs up to 1000."

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.